

Projecte ESIN

ENTREGA

1. Introducció	3
2. Metodologia	4
3. Versions de les Classes i Detalls Tècnics	5
3.1 Classe Word_Toolkit	5
3.1.1 Versió 1 (NO OPTIMITZADA I ÚS DE SORT)	5
3.2 Classe Iter_Subset	7
3.2.1 Versió 1 (Violació de segment)	7
3.2.2 Versió 2 (Canvi en operator++())	10
3.2.3 Versió 3 (Afegida condició especial en operator==())	11
3.3 Classe Diccionari	13
3.3.1 Versió 1 (Trie tradicional)	13
3.3.2 Versió 2 (Trie TST)	13
3.4 Classe Anagrames	15
3.4.1 Versió 1 (Violació de Segment)	15
3.4.2 Versió 2 (Excepció de coma flotant)	18
3.4.3 Versió 3 (free()): invalid size)	19
3.4.4 Versió 4 (Modificació del private)	21
3.5 Classe Obte_Paraules	27
3.5.1 Versió 1	27
3.5.2 Versió 2	27
4. Classes finals	29
4.1 Classe Word_Toolkit Final	29
4.2 Classe Iter_Subset Final	31
4.3 Classe Diccionari Final	35
4.4 Classe Anagrames Final	35
4.5 Classe Obte_Paraules Final	40
5. Conclusions i Lliçons Apreses	41

1. Introducció

El projecte que presentem gira al voltant de la creació d'un conjunt de classes i mòduls dissenyats per abordar, de manera eficient, els reptes dels jocs de paraules populars, com ara Word Challenge. L'objectiu és identificar totes les paraules vàlides d'un diccionari que es poden formar amb un conjunt de lletres donades, optimitzant el temps de resposta i garantint la robustesa del sistema.

Durant el desenvolupament, hem seguit un disseny modular, implementant classes com diccionari, que gestiona eficientment les paraules emmagatzemades, i anagrames, que amplia les funcionalitats per identificar paraules amb un mateix anagrama canònic. Altres components, com `iter_subset`, permeten generar combinacions de lletres, i el mòdul `obte_paraules` integra aquestes eines per oferir una solució completa al problema plantejat.

Una de les nostres prioritats ha estat assegurar que els algorismes i estructures de dades seleccionades fossin adequats per gestionar grans volums d'informació, seguint les especificacions del document oficial. Així mateix, hem tingut cura de documentar el nostre treball, detallant les decisions preses i els motius darrere d'aquestes.

Aquest projecte ha representat una oportunitat per aprofundir en conceptes teòrics i posar-los en pràctica en un context real. A més, ens ha permès desenvolupar habilitats clau com la resolució de problemes, el treball en equip i la capacitat d'adaptar-nos a les limitacions del sistema i els requisits del projecte. El resultat és un conjunt de mòduls que, més enllà del joc específic, poden ser reutilitzats i ampliat en altres contextos similars.

2. Metodologia

Per al desenvolupament d'aquest projecte, hem seguit una metodologia estructurada i progressiva, assegurant-nos de completar i verificar cada component abans de passar al següent. Aquesta aproximació ens ha permès garantir la coherència, l'eficiència i l'alta qualitat de la implementació final.

1. Desenvolupament seqüencial dels mòduls:

Hem començat pel mòdul més bàsic, `word_toolkit`, que proporciona funcions auxiliars essencials per a la resta de classes. A continuació, vam abordar `iter_subset`, una eina clau per generar subconjunts en ordre lexicogràfic. Després, vam implementar diccionari, la classe principal que gestiona les paraules, seguit de anagrames, que amplia les funcionalitats per treballar amb anagrames de manera eficient. Finalment, vam completar el desenvolupament amb el mòdul `obte_paraules`, que integra totes les peces per obtenir paraules vàlides segons les restriccions definides.

2. Focalització en l'eficiència:

En cada etapa, ens hem esforçat per implementar solucions que fossin no només correctes sinó també òptimes en termes d'eficiència. Hem analitzat amb detall les estructures de dades i els algorismes utilitzats, buscant alternatives que permetessin una execució ràpida i una gestió eficient dels recursos.

3. Verificació exhaustiva:

Un cop implementat cada mòdul, hem comprovat el seu funcionament utilitzant els jocs de prova públics proporcionats. Quan aquests es superaven, dissenyàvem jocs de prova privats, amb casos específics i més complexos, per assegurar-nos que el mòdul funcionava de manera impecable en qualsevol situació possible.

4. Enfocament iteratiu:

Si trobàvem qualsevol error o àrea de millora, tornàvem a revisar i ajustar la implementació abans de continuar amb el següent mòdul. Aquest procés iteratiu ens ha permès detectar i solucionar problemes de manera precoç, evitant acumulacions de problemes en fases posteriors.

Aquest enfocament metòdic i centrat en la qualitat ha estat clau per obtenir un resultat final robust i ben optimitzat, preparat per superar tant els jocs de prova públics com els privats amb èxit.

3. Versions de les Classes i Detalls Tècnics

En aquest apartat, explicarem detalladament el procés de desenvolupament de cadascuna de les classes del projecte. Per a cada classe, presentarem les diferents versions que hem anat implementant, descrivint els canvis que hem realitzat per solucionar problemes, afegir funcionalitats o optimitzar el codi. A través d'aquest procés, mostrarem com hem anat millorant progressivament fins a arribar a una versió final robusta i funcional.

Aquest enfocament no només permet entendre el producte final, sinó també el camí recorregut i les decisions preses durant el desenvolupament, oferint una visió completa de la nostra metodologia de treball i del nostre aprenentatge continu.

3.1 Classe Word_Toolkit

La classe Word_Toolkit és el punt de partida del nostre projecte, i la seva implementació inicial ha estat relativament senzilla. Els problemes principals han estat aconseguir que les funcions fossin prou eficients i adaptades a les necessitats específiques del projecte. En aquest apartat, explorarem les diferents versions desenvolupades per millorar-ne el disseny i el rendiment fins a arribar a la versió final.

3.1.1 Versió 1 (NO OPTIMITZADA I ÚS DE SORT)

En aquesta primera versió, vam implementar les funcions principals de manera funcional, però amb mètodes menys eficients i més rudimentaris. Un punt clau és l'ús del mètode sort per ordenar les cadenes lexicogràficament, que té un cost de $\Theta(n \log n)$, en lloc d'una solució basada en comptadors, com es fa a la versió final.

Funcions implementades en aquesta versió:

- `es_canonic`: Aquesta funció no va patir canvis significatius al llarg del desenvolupament. Verifica si les lletres d'una cadena estan ordenades lexicogràficament.
- `anagrama_canonic`: Aquí és on la versió inicial es diferencia més. Per ordenar lexicogràficament, es va utilitzar un enfocament bàsic basat en l'algorisme de sort, que no aprofita estructures més eficients com arrays de comptadors.
- `mes_frequent`: Tot i ser funcional, aquesta versió utilitza bucles imbricats per comptar les freqüències, cosa que fa que el cost sigui més elevat. No es fa ús d'optimitzacions com arrays de comptadors per agilitzar el procés.

Aquestes limitacions ens van portar a explorar solucions més eficients en versions posteriors, que redueixen significativament el cost computacional de les funcions.

```
C/C++  
#include "word_toolkit.hpp"  
#include <algorithm>
```

```

/* Pre: Cert
Post: Retorna cert si i només si les lletres de l'string s estan
en ordre lexicogràfic ascendent. */
bool word_toolkit::es_canonic(const string& s) throw() { // cost  $\Theta(n)$ 
    bool canonic = true;
    for (unsigned int i = 1; i < s.size(); i++) {
        if (s[i] < s[i - 1]) {
            canonic = false;
            break;
        }
    }
    return canonic;
}

/* Pre: Cert
Post: Retorna l'anagrama canònic de l'string s.
Veure la classe anagrames per saber la definició d'anagrama canònic. */
string word_toolkit::anagrama_canonic(const string& s) throw() { // cost  $\Theta(n \log n)$ 
    string letras_validas;
    for (unsigned int i = 0; i < s.size(); i++) {
        if (s[i] >= 'A' && s[i] <= 'Z') { // Solo incluye letras entre 'A' y 'Z'
            letras_validas += s[i];
        }
    }
    sort(letras_validas.begin(), letras_validas.end()); // Ordena lexicográficamente
    return letras_validas;
}

/* Pre: L és una llista no buida de paraules formades exclusivament
amb lletres majúscules de la 'A' a la 'Z' (excloses la 'Ñ', 'Ç',
majúscules accentuades, ...).
Post: Retorna el caràcter que no apareix a l'string excl i és
el més freqüent en la llista de paraules L.
En cas d'empat, es retornaria el caràcter alfabèticament menor.
Si l'string excl inclou totes les lletres de la 'A' a la 'Z' es
retorna el caràcter '\0', és a dir, el caràcter de codi ASCII 0. */
char word_toolkit::mes_frecuent(const string& excl, const list<string>& L) throw() {
    // cost  $\Theta(n + m \cdot k)$ 
    // Contadores para las frecuencias
    int max_frecuencia = 0;
    char letra_mas_frecuente = '\0';

    // Probar cada letra del alfabeto
    for (char letra = 'A'; letra <= 'Z'; ++letra) {
        // Comprobar si la letra está en el string excl (no la contamos si está
        // excluida)
        bool excluida = false;
        for (unsigned int i = 0; i < excl.size(); ++i) {
            if (excl[i] == letra) {
                excluida = true;
                break; // Salimos del bucle si encontramos la letra en excl
            }
        }
    }
}

```

```

// Si la letra está excluida, no la contamos
if (!excluida) {
    // Contar cuántas veces aparece la letra en las palabras de L
    int frecuencia_actual = 0;
    for (list<string>::const_iterator it = L.begin(); it != L.end(); ++it) {
        const std::string& palabra = *it;
        for (unsigned int j = 0; j < palabra.size(); ++j) {
            if (palabra[j] == letra) {
                ++frecuencia_actual;
            }
        }
    }

    // Actualizar si es más frecuente o alfabéticamente menor en caso de empate
    if (frecuencia_actual > max_frecuencia or (frecuencia_actual ==
max_frecuencia and letra < letra_mas_frecuente)) {
        max_frecuencia = frecuencia_actual;
        letra_mas_frecuente = letra;
    }
}

return letra_mas_frecuente;
}

```

3.2 Classe Iter_Subset

La classe Iter_Subset ha estat una de les més complexes a desenvolupar, ja que ens ha requerit implementar fins a tres versions abans d'arribar a la versió final. El principal desafiament ha estat fixar els errors que ens han anat sortint i trobar una manera eficient de generar tots els subconjunts possibles d'un conjunt donat de lletres, mantenint un ús òptim de memòria i temps d'execució.

Al llarg del procés, hem treballat per reduir redundàncies i optimitzar el codi, així com per fer-lo més modular i fàcil de mantenir. Aquesta classe ha estat clau per aconseguir generar totes les combinacions necessàries de manera robusta i escalable.

3.2.1 Versió 1 (Violació de segment)

En aquesta primera versió de Iter_Subset, es va desenvolupar un iterador per generar tots els subconjunts de k elements d'un conjunt de n elements. Inicialment, la classe semblava funcional, però aviat es van identificar diversos errors crítics:

Quan $k > n$, la classe no gestionava adequadament aquesta situació. Això provocava una violació de segment en intentar accedir a elements no inicialitzats de `_current`.

El mètode `operator++` no controlava correctament el final dels subconjunts. En alguns casos, intentava avançar més enllà de l'últim subconjunt vàlid, alterant l'estat de `_end`.

Hi havia una manca de validacions robustes per gestionar situacions límit, com iteracions amb conjunts buits o valors incorrectes de k i n .

Aquestes deficiències van fer evident la necessitat de redissenyar la classe. A les següents versions, es van aplicar millores per fer-la més segura i funcional, resolent els errors identificats.

```
C/C++
#include "iter_subset.hpp"

/* Pre: Cert
Post: Construeix un iterador sobre els subconjunts de k elements
de {1, ..., n}; si k > n no hi ha res a recórrer. */
iter_subset::iter_subset(nat n, nat k) throw(error) { // cost  $\Theta(k)$ 
    _n = n;
    _k = k;
    _end = false;
    if (k > n) {
        _end = true; // No hay subconjuntos posibles
    } else {
        _current.resize(k);
        for (nat i = 0; i < k; ++i) {
            _current[i] = i + 1; // Inicializar el primer subconjunto
        }
    }
}

/* Tres grans. Constructor per còpia, operador d'assignació i destructor. */
iter_subset::iter_subset(const iter_subset& its) throw(error) { // cost  $\Theta(k)$ 
    _n = its._n;
    _k = its._k;
    _current = its._current;
    _end = its._end;
}

iter_subset& iter_subset::operator=(const iter_subset& its) throw(error) { // cost  $\Theta(k)$ 
    if (this != &its) {
        _n = its._n;
        _k = its._k;
        _current = its._current;
        _end = its._end;
    }
    return *this;
}

iter_subset::~iter_subset() throw() {}

/* Pre: Cert
Post: Retorna cert si l'iterador ja ha visitat tots els subconjunts
de k elements presos d'entre n; o dit d'una altra forma, retorna
cert quan l'iterador apunta a un subconjunt sentinella fictici
que queda a continuació de l'últim subconjunt vàlid. */
```



```

bool iter_subset::end() const throw() { // cost 0(1)
    return _end;
}

/* Operador de desreferència.
Pre: Cert
Post: Retorna el subconjunt apuntat per l'iterador;
llança un error si l'iterador apunta al sentinella. */
subset iter_subset::operator*() const throw(error) { // cost 0(1)
    if (_end) {
        throw error(IterSubsetIncorr);
    }
    return _current;
}

/* Operador de preincremento.
Pre: Cert
Post: Avança l'iterador al següent subconjunt en la seqüència i el retorna;
no es produeix l'avançament si l'iterador ja apuntava al sentinella. */
iter_subset& iter_subset::operator++() throw() {
    if (_end) {
        return *this;
    }

    // Avanzar al siguiente subconjunto lexicográfico
    nat i = _k - 1;
    while (i >= 0 && _current[i] == _n - _k + i + 1) {
        --i;
    }

    if (i < 0) {
        _end = true; // No hay más subconjuntos
    } else {
        ++_current[i];
        for (nat j = i + 1; j < _k; ++j) {
            _current[j] = _current[j - 1] + 1;
        }
    }

    return *this;
}

/* Operador de postincremento.
Pre: Cert
Post: Avança l'iterador al següent subconjunt en la seqüència i retorna el seu valor
previ; no es produeix l'avançament si l'iterador ja apuntava al sentinella. */
iter_subset iter_subset::operator++(int) throw() {
    iter_subset temp(*this);
    ++(*this);
    return temp;
}

/* Operadors relacionals. */
bool iter_subset::operator==(const iter_subset& c) const throw() { // cost 0(k)

```

```

    return _n == c._n && _k == c._k && _current == c._current && _end == c._end;
}

bool iter_subset::operator!=(const iter_subset& c) const throw() { // cost  $\theta(k)$ 
    return !(*this == c);
}

```

3.2.2 Versió 2 (Canvi en operator++())

En aquesta segona versió de `Iter_Subset`, es va abordar el problema principal de la versió anterior: l'`operator++()` no gestionava correctament l'avanç cap al següent subconjunt en certs casos límit. A més, es va implementar una millor gestió per als casos on $k > n$ i es van afegir validacions més estrictes per assegurar la coherència de l'estat intern de l'iterador. És a dir:

1. Correcció de l'`operator++()`:
 - Ara verifica si el subconjunt actual és l'últim possible comparant-lo amb un subconjunt final calculat (`lastSubset`).
 - Si s'ha arribat al final, s'estableix el flag `_end` a `true`.
2. Millor gestió del cas $k > n$:
 - Quan $k > n$, el vector `_current` es buida explícitament i es marca `_end` com a cert, evitant errors posteriors.
3. Reinicialització segura dels elements posteriors:
 - En incrementar un element del subconjunt, els valors següents es reinicien perquè continuïn ordenats lexicogràficament.

Aquesta versió va solucionar els errors de la versió anterior i va assegurar que l'iterador pogués avançar correctament a través dels subconjunts sense caure en incoherències o violacions de segment. Tot i això, es va considerar que encara podia ser optimitzada i simplificada en una versió posterior.

Funcions que han canviat:

```

C/C++
iter_subset::iter_subset(nat n, nat k) throw(error) { // cost  $\theta(k)$ 
    _n = n;
    _k = k;
    _end = false;
    if (k > n) {
        _end = true;
        _current.clear();
    }
}

```

```

    } else {
        _current.resize(k);
        for (nat i = 0; i < k; ++i) {
            _current[i] = i + 1;
        }
    }
}

iter_subset& iter_subset::operator++() throw() {
    if (_end) {
        return *this;
    }

    subset lastSubset;
    for (nat i = _n - _k; i < _n; ++i) {
        lastSubset.push_back(i + 1);
    }

    if (_current == lastSubset) {
        _end = true;
    } else {
        for (nat i = _k - 1; i >= 0; --i) {
            if (_current[i] < _n - (_k - 1 - i)) {
                ++_current[i];
                for (nat j = i + 1; j < _k; ++j) {
                    _current[j] = _current[j - 1] + 1;
                }
                return *this;
            }
        }
        _end = true;
    }

    return *this;
}

```

3.2.3 Versió 3 (Afegida condició especial en operator==())

En aquesta versió, es va afegir una condició especial a l'operador operator==() per gestionar correctament els casos on $k = 0$. Això permet comparar iteradors de manera coherent fins i tot quan no hi ha subconjunts possibles.

Canvis principals:

- Condició especial en operator==(): S'ha afegit una comprovació per comparar correctament els iteradors quan $k = 0$.
- Millora en la comparació: Ara els iteradors amb subconjunts buits es comparen de manera fiable.

Per tant, resumint, es resolen els casos límit amb subconjunts buits i es millora la coherència del comportament de l'iterador.

Funció que ha canviat:

```
C/C++
bool iter_subset::operator==(const iter_subset& c) const throw() { // cost  $\Theta(k)$ 
    if (_n != c._n and _k == 0 and c._k == 0 and _current == c._current and _end ==
c._end) {
        return true;
    }
    return _n == c._n and _k == c._k and _current == c._current and _end == c._end;
}
```

3.3 Classe Diccionari

3.3.1 Versió 1 (Trie tradicional)

En aquesta primera versió de la classe Diccionari, vam implementar una solució utilitzant un Trie tradicional. Aquesta estructura de dades emprava un array de 26 punters per node, cadascun representant una lletra de l'alfabet. Encara que la implementació era senzilla i intuïtiva per a gestionar paraules i prefixos, presentava alguns problemes.

1. Alt consum de memòria:
Cada node del Trie contenia un array fix de 26 punters, independentment del nombre real de fills. Això significava que molts nodes tenien gran part de l'array buit, especialment en diccionaris amb poques paraules o amb moltes paraules amb prefixos comuns. Això feia que la implementació fos molt poc eficient en termes d'espai.
2. Dificultat gestió de memòria:
Encara que l'ús d'un array fix simplificava l'accés als fills, complicava la gestió de la memòria. Copiar i alliberar correctament els nodes del Trie era un procés més laboriós i susceptible a errors, especialment en el destructor i en el mètode de còpia. Això podia derivar en violacions de segment (segmentation faults) si no es gestionaven adequadament els punters.
3. Rigidesa de l'estructura:
Utilitzar un array fix de 26 punters feia que l'estructura fos molt poc adaptable. Per exemple, si es volia suportar un conjunt de caràcters més ampli (com accents, encara que no fos-hi el cas), s'hauria de redissenyar completament l'array.

C/C++
poner codigo

3.3.2 Versió 2 (Trie TST)

En aquesta segona versió de la classe Diccionari, es van introduir millores importants que van solucionar problemes de la primera implementació i van optimitzar l'eficiència de l'estructura. Un dels canvis principals va ser la substitució de l'estructura de Trie tradicional per un Trie Ternari de Cerca (TST), cosa que va reduir el consum de memòria i va fer el codi més escalable.

Principals canvis i millores:

1. Substitució del Trie tradicional pel TST:

Abans, cada node del Trie contenia un array fix de 26 punters, fet que causava un gran consum de memòria, especialment en casos amb conjunts petits de caràcters.

Ara, el Trie s'ha substituït per un Trie Ternari de Cerca (TST), on cada node només conté tres punters (`_esq`, `_cen`, `_dre`), que representen la relació dels caràcters (menors, iguals o majors).

Impacte del canvi:

Aquest canvi redueix dràsticament el consum de memòria en diccionaris amb moltes paraules o amb caràcters rars, ja que només s'assigna memòria per als nodes necessaris.

2. Optimització de la gestió de la memòria:

A la Versió 1, l'ús d'un array fix implicava una gestió complexa dels punters, cosa que podia provocar errors de segmentació (segmentation faults) durant les operacions de còpia o esborrat.

Ara, gràcies al TST, només s'han de gestionar tres punters per node, fent el codi més robust i reduint la probabilitat d'errors.

Impacte del canvi:

Millora la seguretat i fiabilitat del codi, fent-lo menys susceptible a problemes de memòria.

3. Flexibilitat per treballar amb altres conjunts de caràcters:

A la Versió 1, l'array fix limitava l'estructura als caràcters de l'alfabet anglès (26 lletres).

Ara, el TST permet comparar directament caràcters de qualsevol conjunt (per exemple, caràcters amb accents o d'altres llengües).

Impacte del canvi:

Fa que l'estructura sigui més adaptable a diferents idiomes o conjunts de dades.

Funcions que han canviat:

1. Inserció d'elements:

Abans: Basada en l'assignació de punters dins d'un array de 26 posicions.

Ara: Utilitza un Trie Ternari amb comparacions entre caràcters per decidir la direcció del node següent (`_esq`, `_cen`, `_dre`).

C/C++

POSA CODI

2. Cerca de paraules:

Abans: Recorria l'array fix de punters segons l'índex del caràcter.

Ara: Es basa en una cerca binària dins del TST, seguint l'ordre relatiu dels caràcters.

C/C++
POSA CODI

3. Destructor i gestió de memòria:

Abans: Recorregut manual de l'array per alliberar els punters.

Ara: Simplificació del destructor gràcies a la menor quantitat de punters en cada node.

C/C++
POSA CODI

3.4 Classe Anagrames

3.4.1 Versió 1 (Violació de Segment)

En aquesta primera versió de la classe Anagrames, vam implementar una solució senzilla per gestionar anagrames amb una taula de dispersió. Utilitzava diverses llibreries externes com `stdexcept`, `algorithm` i `climits`. Però aquesta versió presentava un error de violació de segment degut a una mala gestió de la memòria, especialment quan copiava i alliberava els nodes.

Bàsicament, els problemes eren que la violació de segment es produïa per una gestió incorrecta dels punters a l'hora de copiar i esborrar els nodes. I que vam fer servir més llibreries externes de les necessàries, cosa que complicava la solució i afegia dependències innecessàries.

En resum, aquesta versió era una implementació inicial senzilla, però amb problemes greus de gestió de memòria i dependències externes.

```
C/C++
#include "anagrames.hpp"
#include "word_toolkit.hpp"
#include <stdexcept>
#include <algorithm>
#include <climits>

anagrames::anagrames() throw(error) : _M(101), _quants(0) {
    _taula.resize(_M, nullptr);
}
```

```
anagrames::anagrames(const anagrames& A) throw(error) : _M(A._M), _quants(A._quants) {
    _taula.resize(_M, nullptr);
    for (size_t i = 0; i < _M; ++i) {
        if (A._taula[i]) {
            _taula[i] = copia_nodes_anagrames(A._taula[i]);
        }
    }
}

anagrames& anagrames::operator=(const anagrames& A) throw(error) {
    if (this != &A) {
        for (size_t i = 0; i < _M; ++i) {
            esborra_nodes(_taula[i]);
        }

        _M = A._M;
        _quants = A._quants;
        _taula.resize(_M, nullptr);

        for (size_t i = 0; i < _M; ++i) {
            if (A._taula[i]) {
                _taula[i] = copia_nodes_anagrames(A._taula[i]);
            }
        }
    }
    return *this;
}

anagrames::~~anagrames() throw() {
    for (size_t i = 0; i < _M; ++i) {
        esborra_nodes(_taula[i]);
    }
}

void anagrames::insereix(const string& p) throw(error) {
    string anagrama = word_toolkit::anagrama_canonic(p);
    size_t index = h(anagrama) % _M;

    node_hash* actual = _taula[index];
    while (actual) {
        if (actual->_clau == anagrama) {
            if (find(actual->_paraules.begin(), actual->_paraules.end(), p) ==
                actual->_paraules.end()) {
                actual->_paraules.push_back(p);
            }
            return;
        }
        actual = actual->_seg;
    }

    node_hash* nuevo = new node_hash(anagrama, {p}, _taula[index]);
    _taula[index] = nuevo;
}
```



```

    ++_quants;
}

void anagrames::mateix_anagrama_canonic(const string& a, list<string>& L) const
throw(error) {
    if (!word_toolkit::es_canonic(a)) {
        throw error(NoEsCanonic);
    }

    size_t index = h(a) % _M;
    node_hash* actual = _taula[index];

    while (actual) {
        if (actual->_clau == a) {
            L = actual->_paraules;
            return;
        }
        actual = actual->_seg;
    }

    L.clear();
}

long anagrames::h(const string& clau) const {
    long valor = 0;
    for (char c : clau) {
        valor = (MULT * valor + c) % LONG_MAX;
    }
    return valor;
}

void anagrames::esborra_nodes(node_hash* p) {
    while (p) {
        node_hash* temp = p;
        p = p->_seg;
        delete temp;
    }
}

anagrames::node_hash* anagrames::copia_nodes_anagrames(const node_hash* p) {
    if (!p) return nullptr;

    node_hash* nuevo = new node_hash(p->_clau, p->_paraules, nullptr);
    node_hash* actual = nuevo;

    while (p->_seg) {
        p = p->_seg;
        actual->_seg = new node_hash(p->_clau, p->_paraules, nullptr);
        actual = actual->_seg;
    }

    return nuevo;
}

```

3.4.2 Versió 2 (Excepció de coma flotant)

En aquesta versió es van introduir algunes millores per gestionar millor els anagrames. Un dels canvis més importants va ser l'actualització de la funció `h()` (funció hash) per garantir que l'índex sempre fos un valor no negatiu. Tot i així, aquesta versió presentava un problema d'excepció de coma flotant en determinats casos a causa d'errors en la gestió dels valors de la funció hash, que no es validaven correctament.

Principals canvis i millores:

1. Gestió de l'excepció de coma flotant a la funció `h()`:
 - Abans, la funció hash podia generar índexs negatius. Ara, es corregeix per assegurar-se que l'índex sempre sigui no negatiu.
 - S'ha modificat la línia final de la funció `h()` per garantir que el valor retornat sigui sempre no negatiu, utilitzant `valor >= 0 ? valor : -valor`.
2. Manteniment de la lògica de taula de dispersió:
 - La lògica d'inserir nous anagrames i de recuperar-los a partir de l'anagrama canònic es manté, millorant només la seguretat de la gestió d'índexs.

Aquesta versió va millorar la gestió d'índexs i va corregir l'excepció de coma flotant, però la gestió de la memòria i les operacions continuaven sent similars a la versió anterior.

Funcions que han canviat:

```
C/C++
void anagrames::insereix(const string& p) throw(error) {
    string anagrama = word_toolkit::anagrama_canonic(p);
    size_t index = h(anagrama) % _M;

    if (_taula[index] == nullptr) {
        _taula[index] = new node_hash(anagrama, {p}, nullptr);
        ++_quants;
        return;
    }

    node_hash* actual = _taula[index];
    while (actual) {
        if (actual->_clau == anagrama) {
            if (find(actual->_paraules.begin(), actual->_paraules.end(), p) ==
                actual->_paraules.end()) {
                actual->_paraules.push_back(p);
            }
            return;
        }
        actual = actual->_seg;
    }
}
```

```

node_hash* nuevo = new node_hash(anagrama, {p}, _taula[index]);
_taula[index] = nuevo;
++quants;
}

void anagrames::mateix_anagrama_canonic(const string& a, list<string>& L) const
throw(error) {
    if (!word_toolkit::es_canonic(a)) {
        throw error(NoEsCanonic);
    }

    size_t index = h(a) % _M;

    if (_taula[index] == nullptr) {
        L.clear();
        return;
    }

    node_hash* actual = _taula[index];
    while (actual) {
        if (actual->_clau == a) {
            L = actual->_paraules;
            return;
        }
        actual = actual->_seg;
    }

    L.clear();
}

long anagrames::h(const string& clau) const {
    long valor = 0;
    for (char c : clau) {
        valor = (MULT * valor + c) % LONG_MAX;
    }
    return (valor >= 0 ? valor : -valor);
}

```

3.4.3 Versió 3 (free(): invalid size)

En aquesta versió, es van fer millores importants per evitar errors com els índexs negatius a la funció hash. La funció es va actualitzar perquè sempre retorni un índex no negatiu, la qual cosa va evitar l'error de “coma flotant” que apareixia a la versió anterior. A més, es va afegir una validació per assegurar-se que no es passessin paraules buides a la funció, millorant la seguretat del codi.

La lògica de la taula de dispersió i les operacions per inserir i recuperar paraules es va mantenir bastant similar a la versió anterior, però amb més controls per evitar errors. Tot i aquestes millores, la gestió de la memòria continuava sent un punt feble, ja que no es va modificar la manera com es gestionaven els nodes i els recursos. Aquesta versió va millorar

la seguretat i va evitar els problemes de la versió anterior, però encara hi havia espai per a optimitzacions en altres àrees.

```
C/C++
void anagrames::insereix(const string& p) throw(error) {
    if (p.empty()) {
        throw error(NoEsCanonic);
    }

    string anagrama = word_toolkit::anagrama_canonic(p);
    size_t index = (_M > 0) ? h(anagrama) % _M : 0;

    if (_taula[index] == nullptr) {
        _taula[index] = new node_hash(anagrama, {p}, nullptr);
        ++_quants;
        return;
    }

    node_hash* actual = _taula[index];
    while (actual) {
        if (actual->_clau == anagrama) {
            if (find(actual->_paraules.begin(), actual->_paraules.end(), p) ==
                actual->_paraules.end()) {
                actual->_paraules.push_back(p);
            }
            return;
        }
        actual = actual->_seg;
    }

    node_hash* nuevo = new node_hash(anagrama, {p}, _taula[index]);
    _taula[index] = nuevo;
    ++_quants;
}

void anagrames::mateix_anagrama_canonic(const string& a, list<string>& L) const
throw(error) {
    if (!word_toolkit::es_canonic(a)) {
        throw error(NoEsCanonic);
    }

    size_t index = (_M > 0) ? h(a) % _M : 0;

    if (_taula[index] == nullptr) {
        L.clear();
        return;
    }

    node_hash* actual = _taula[index];
    while (actual) {
        if (actual->_clau == a) {
            L = actual->_paraules;
            return;
        }
    }
}
```

```
    }
    actual = actual->_seg;
}

L.clear();
}

long anagrames::h(const string& clau) const {
    if (clau.empty()) {
        throw error(NoEsCanonic);
    }

    long valor = 0;
    for (char c : clau) {
        valor = (MULT * valor + c) % LONG_MAX;
    }
    return (valor >= 0 ? valor : -valor);
}
```

3.4.4 Versió 4 (Modificació del private)

En la versió 4, ens vam trobar amb molts errors relacionats amb la secció private de les versions anteriors, que dificultaven que la classe compilés correctament. Com a resultat, vam decidir canviar completament aquesta part del codi i refactorar la classe des de zero. L'objectiu era simplificar la gestió dels elements privats per evitar els problemes que havíem tingut fins aquell moment.

Després de provar diverses solucions, finalment vam aconseguir una implementació que feia que la classe funcionés bé, però la gestió del private era força diferent, i la taula de dispersió no estava implementada com la volíem. Així que tot i que la classe ara funcionava correctament, no teníem la taula de dispersió de la mateixa manera que en les versions anteriors.

Va ser en la versió final quan vam aconseguir integrar la taula de dispersió de manera semblant a les versions 1, 2 i 3, i vam poder mantenir la simplificació de la classe que havíem aconseguit a la versió 4. D'aquesta manera, vam solucionar tots els errors anteriors i vam obtenir una implementació neta i eficient de la classe.

En resum, en la versió 4 vam simplificar la classe i resoldre els errors de la secció private, però va ser només en la versió final quan vam aconseguir integrar la taula de dispersió de manera similar a les versions anteriors.

Archiu anagrames.rep (private) :

```

C/C++
struct node_hash {
    string _k;
    list<string> _v;
    node_hash* _seg;

    node_hash(const string& k) : _k(k), _seg(nullptr) {}
};

class rep {
public:
    node_hash** _taula;
    nat _M;
    nat _quants;
    diccionari dicc;

    nat hash(const string& k) const {
        nat h = 0;
        for (nat i = 0; i < k.size(); ++i) {
            char c = k[i];
            h = (h * 31 + c) % _M;
        }
        return h;
    }
    rep(nat M = 101) {
        _M = M;
        _quants = 0;
        _taula = new node_hash*[_M]();
    }

    ~rep() {
        for (nat i = 0; i < _M; ++i) {
            node_hash* actual = _taula[i];
            while (actual) {
                node_hash* temp = actual;
                actual = actual->_seg;
                delete temp;
            }
        }
        delete[] _taula;
    }
};

rep* _rep;

```

Archiu anagrames.cpp:

```

C/C++
#include "anagrames.hpp"
#include "word_toolkit.hpp"

```

```

void swap(char& a, char& b) {
    char temporal = a;
    a = b;
    b = temporal;
}

int particio(string& s, int esquerra, int dreta) {
    char pivot = s[dreta];
    int i = esquerra - 1;

    for (int j = esquerra; j < dreta; ++j) {
        if (s[j] <= pivot) {
            ++i;
            swap(s[i], s[j]);
        }
    }

    swap(s[i + 1], s[dreta]);
    return i + 1;
}

void quicksort(string& s, int esquerra, int dreta) {
    if (esquerra < dreta) {
        int pivot = particio(s, esquerra, dreta);
        quicksort(s, esquerra, pivot - 1);
        quicksort(s, pivot + 1, dreta);
    }
}

anagrames::anagrames() throw(error) {
    taula = new node*[101]();
    mida = 0;
    quants = 101;
}

anagrames::anagrames(const anagrames& other) throw(error) {
    taula = new node*[other.quants]();
    mida = other.mida;
    quants = other.quants;
    for (nat i = 0; i < quants; ++i) {
        if (other.taula[i]) {
            node* actual = other.taula[i];
            node** this_actual = &taula[i];

            while (actual) {
                *this_actual = new node(actual->clau);
                (*this_actual)->paraules = actual->paraules;
                actual = actual->seg;
                this_actual = &((*this_actual)->seg);
            }
        }
    }
}

```

```

anagrames& anagrames::operator=(const anagrames& other) throw(error) {
    if (this != &other) {
        for (nat i = 0; i < quants; ++i) {
            node* n = taula[i];
            while (n) {
                node* temp = n;
                n = n->seg;
                delete temp;
            }
        }
        delete[] taula;

        taula = new node*[other.quants]();
        mida = other.mida;
        quants = other.quants;

        for (nat i = 0; i < quants; ++i) {
            if (other.taula[i]) {
                node* actual = other.taula[i];
                node** this_actual = &taula[i];

                while (actual) {
                    *this_actual = new node(actual->clau);
                    (*this_actual)->paraules = actual->paraules;
                    actual = actual->seg;
                    this_actual = &((*this_actual)->seg);
                }
            }
        }
        return *this;
    }
}

anagrames::~~anagrames() {
    for (nat i = 0; i < quants; ++i) {
        node* n = taula[i];
        while (n != nullptr) {
            node* temp = n;
            n = n->seg;
            delete temp;
        }
        taula[i] = nullptr;
    }
    delete[] taula;
    taula = nullptr;
}

void anagrames::insereix(const string& p) throw(error) {
    diccionari::insereix(p);

    string p_ordenada = p;
    quicksort(p_ordenada, 0, p_ordenada.size() - 1);
}

```



```

    nat index = h(p_ordenada);
    node* n = taula[index];

    while (n != nullptr) {
        if (n->clau == p_ordenada) {
            for (list<string>::iterator it = n->paraules.begin(); it !=
n->paraules.end(); ++it) {
                if (*it == p) return;
            }
            n->paraules.push_back(p);
            return;
        }
        n = n->seg;
    }

    node* nou_node = new node(p_ordenada);
    nou_node->paraules.push_back(p);
    nou_node->seg = taula[index];
    taula[index] = nou_node;
    ++mida;

    if (mida > quants * 0.75) redispersio();
}

void anagrames::mateix_anagrama_canonic(const string& a, list<string>& L) const
throw(error) {
    if (!word_toolkit::es_canonic(a)) throw error(NoEsCanonic);

    for (list<string>::iterator it = L.begin(); it != L.end(); ) {
        string it_ordenada = *it;
        quicksort(it_ordenada, 0, it_ordenada.size() - 1);
        if (it_ordenada != a) {
            it = L.erase(it);
        } else {
            ++it;
        }
    }

    nat index = h(a);
    node* n = taula[index];

    while (n != nullptr) {
        if (n->clau == a) {
            for (list<string>::iterator it_paula = n->paraules.begin(); it_paula !=
n->paraules.end(); ++it_paula) {
                bool trobat = false;
                for (list<string>::iterator it_existent = L.begin(); it_existent !=
L.end(); ++it_existent) {
                    if (*it_existent == *it_paula) {
                        trobat = true;
                        break;
                    }
                }
                if (!trobat) {

```

```

        L.push_back(*it_paraula);
    }
}
L.sort();
return;
}
n = n->seg;
}
L.clear();
}
nat anagrames::h(const string& clau) const {
    nat hash = 0;
    for (nat i = 0; i < clau.size(); ++i) {
        hash = (hash * 31 + clau[i]) % quants;
    }
    return hash;
}

void anagrames::redispersio() {
    nat nou_quants = quants * 2;
    node** nova_taula = new node*[nou_quants]();

    for (nat i = 0; i < quants; ++i) {
        node* n = taula[i];
        while (n != nullptr) {
            node* temp = n;
            n = n->seg;

            nat nou_index = 0;
            for (size_t j = 0; j < temp->clau.size(); ++j) {
                nou_index = (nou_index * 31 + temp->clau[j]) % nou_quants;
            }

            temp->seg = nova_taula[nou_index];
            nova_taula[nou_index] = temp;
        }
    }

    delete[] taula;
    taula = nova_taula;
    quants = nou_quants;
}

```

3.5 Classe Obte_Paraules

3.5.1 Versió 1

En aquesta primera versió del mòdul obte_paraules, vam implementar una solució senzilla per generar totes les paraules possibles amb k lletres o més a partir d'un string. La implementació utilitza dos classes externes: iter_subset, per gestionar subconjunts, word_toolkit, per treballar amb anagrames. Aquesta versió funciona adequadament per als casos bàsics, però presenta diverses mancances.

1. Les paraules es copiaven a la llista final sense comprovar si ja existien, cosa que generava duplicats en el resultat.
2. No es garantia cap ordre en les paraules generades. Això no complia amb la condició de tenir-les ordenades per longitud i, dins de cada longitud, alfabèticament.
3. La llista principal de resultats i les llistes temporals es gestionaven sense optimitzar el recorregut ni l'ús de recursos. Això podia resultar en un ús innecessari de memòria.
4. Hi havia certa duplicació de funcionalitat entre el primer i el segon mètode, cosa que feia que el codi fos més llarg i difícil de mantenir.

C/C++

3.5.2 Versió 2

En la versió 2 del mòdul obte_paraules, hem introduït una sèrie de millores significatives per resoldre diversos problemes que vam identificar en la versió 1 i per optimitzar el comportament general de les funcions. Durant el desenvolupament de la versió anterior, vam detectar algunes limitacions i aspectes que podien ser millorats, i per això vam centrar-nos en millorar l'eficiència, la gestió de memòria, la coherència en l'ordenació dels resultats i la gestió de duplicats. Aquestes modificacions no només han fet que el codi sigui més robust, sinó també més eficient i fàcil de mantenir.

Funcions que han canviat:

- Es comprova si cada paraula ja existeix a la llista abans d'afegir-la. Això garanteix que la llista final només contingui elements únics.

C/C++

- Les paraules es col·loquen a la llista principal en la posició adequada. Això s'aconsegueix utilitzant una comparació de longitud i ordre alfabètic durant la inserció.

C/C++

- Ara, les paraules es processen i eliminen de les llistes temporals a mesura que es treballa amb elles. Aquesta optimització redueix el consum de memòria i fa que el codi sigui més eficient.

C/C++

- S'ha mantingut l'estructura bàsica però s'ha fet més clara i cohesionada. Ara, el segon mètode reutilitza de manera més òptima la funcionalitat del primer mètode. Els bucles i la gestió de la llista principal són més coherents entre les dues funcions.

C/C++

4. Classes finals

Aquestes són les classes finals i definitives del nostre projecte, després de tot el treball fet en les diferents versions. Hem estat provant, millorant i ajustant cada classe per fer-les més eficients, segures i fàcils de fer servir. Creiem que aquestes versions són les millors, ja que hem aconseguit corregir els errors i optimitzar el rendiment per tal que tot funcioni de manera més fluida. Aquests són els resultats de tot el que hem après durant el procés.

4.1 Classe Word_Toolkit Final

Aquest és el codi definitiu de la classe Word_Toolkit, després de diverses proves i optimitzacions. Aquesta classe proporciona funcions útils per treballar amb cadenes de text, com ordenar les lletres d'una paraula, verificar si una paraula està en ordre lexicogràfic ascendent, o trobar la lletra més freqüent d'una llista de paraules.

Les funcions principals són:

1. `es_canonic`: Comprova si les lletres d'un string estan en ordre lexicogràfic ascendent.
2. `ordenar_lexicograficament`: Ordena les lletres d'una paraula de manera ascendent utilitzant un comptatge de freqüències.
3. `anagrama_canonic`: Retorna l'anagrama canònic d'una paraula, és a dir, les seves lletres ordenades.
4. `mes_frequent`: Troba la lletra més freqüent en una llista de paraules, exclouent aquelles que es passin com a paràmetre.

En aquesta versió final, hem millorat el rendiment i la seguretat del codi, mantenint-lo clar i eficient. Hem aconseguit optimitzar els algorismes per garantir una bona eficiència, fins i tot amb llistes de paraules grans, i el codi es manté fàcil d'utilitzar i per integrar en altres parts del projecte.

```
C/C++
#include "word_toolkit.hpp"

/* Pre: Cert
   Post: Retorna cert si i només si les lletres de l'string s estan
   en ordre lexicogràfic ascendent. */
bool word_toolkit::es_canonic(const string& s) throw() { // cost  $\Theta(n)$ 
    for (unsigned int i = 1; i < s.size(); i++) {
        if (s[i] < s[i - 1]) { // Compara cada lletra amb l'anterior per assegurar-se
            que l'ordre és ascendent
        }
    }
}
```

```

        return false;
    }
    }
    return true;
}

/* Pre: Cert
   Post: Ordena el string s lexicogràficament (alfabèticament ascendent). */
void ordenar_lexicograficament(string& paraula) { // cost  $\Theta(n)$ 
    int comptador[26] = {0}; // Array per comptar la freqüència de cada lletra majúscula

    // Comptem la freqüència de cada lletra
    for (unsigned int i = 0; i < paraula.size(); ++i) {
        if (paraula[i] >= 'A' && paraula[i] <= 'Z') {
            ++comptador[paraula[i] - 'A'];
        }
    }

    unsigned int index = 0;
    // Reconstruïm el string ordenat basant-nos en la freqüència de cada lletra
    for (int i = 0; i < 26; ++i) {
        while (comptador[i] > 0) {
            paraula[index++] = 'A' + i;
            --comptador[i];
        }
    }

    paraula = paraula.substr(0, index); // Retallem el string fins a l'última lletra vàlida
}

/* Pre: Cert
   Post: Retorna l'anagrama canònic de l'string s.
   Veure la classe anagrames per saber la definició d'anagrama canònic. */
string word_toolkit::anagrama_canonic(const string& s) throw() { // cost  $\Theta(n)$ 
    string lletres_valides;
    // Filtra només les lletres majúscules de la 'A' a la 'Z'
    for (unsigned int i = 0; i < s.size(); i++) {
        if (s[i] >= 'A' && s[i] <= 'Z') {
            lletres_valides += s[i];
        }
    }
    ordenar_lexicograficament(lletres_valides); // Ordena lexicogràficament les lletres vàlides
    return lletres_valides;
}

/* Pre: L és una llista no buida de paraules formades exclusivament
   amb lletres majúscules de la 'A' a la 'Z' (excloses la 'Ñ', 'Ç', majúscules accentuades, ...).
   Post: Retorna el caràcter que no apareix a l'string excl i és el més freqüent en la llista de paraules L.
   En cas d'empat, es retornaria el caràcter alfabèticament menor.

```

```

    Si l'string excl inclou totes les lletres de la 'A' a la 'Z' es
    retorna el caràcter '\0', és a dir, el caràcter de codi ASCII 0. */
char word_toolkit::mes_frequent(const string& excl, const list<string>& L) throw() {
    // cost  $\Theta(n + m * k + 26) \approx \Theta(n + m * k)$ 
    bool exclosa[26] = {false};
    // Marca les lletres a excl com a excloses
    for (unsigned int i = 0; i < excl.size(); ++i) {
        if (excl[i] >= 'A' && excl[i] <= 'Z') {
            exclosa[excl[i] - 'A'] = true;
        }
    }

    int comptador[26] = {0};

    // Recorre cada paraula de la llista i compta la freqüència de les lletres no
    excloses
    for (list<string>::const_iterator it = L.begin(); it != L.end(); ++it) {
        const string& paraula = *it;
        for (unsigned int j = 0; j < paraula.size(); ++j) {
            char lletra = paraula[j];
            if (lletra >= 'A' && lletra <= 'Z' && !exclosa[lletra - 'A']) {
                ++comptador[lletra - 'A'];
            }
        }
    }

    char lletra_mes_frequent = '\0';
    int max_frequencia = 0;
    // Cerca la lletra més freqüent que no estigui exclosa
    for (char lletra = 'A'; lletra <= 'Z'; ++lletra) {
        if (comptador[lletra - 'A'] > max_frequencia) {
            max_frequencia = comptador[lletra - 'A'];
            lletra_mes_frequent = lletra;
        }
    }

    return lletra_mes_frequent; // Retorna la lletra més freqüent o '\0' si cap
    lletra és vàlida
}

```

4.2 Classe Iter_Subset Final

Aquesta és la versió final de la classe Iter_Subset, que implementa un iterador per generar tots els subconjunts possibles de k elements d'un conjunt de {1, ..., n}. Després de moltes proves i ajustaments, hem aconseguit una versió optimitzada que gestiona correctament la creació i la iteració pels subconjunts.

Les funcions clau d'aquesta classe són:

1. Constructor: Inicialitza l'iterador per generar subconjunts de k elements d'un conjunt de n elements. Si k és més gran que n , no es genera cap subconjunt.
2. Operador ++: Permet avançar l'iterador al següent subconjunt en la seqüència, generant-lo de manera eficient.
3. Operadors de comparació (==, !=): Comprova si dos iteradors apunten al mateix subconjunt o no.
4. Operador *: Desreferència l'iterador, retornant el subconjunt actual al qual apunta.

Aquesta versió ha estat dissenyada per ser eficaç i fàcil de fer servir, aprofitant un mètode incremental per generar subconjunts. Hem optimitzat els operadors i la gestió de la memòria per assegurar que el codi sigui ràpid, robust i amb un rendiment elevat, fins i tot per a conjunts grans.

Archiu iter_subset.rep:

```
C/C++
nat _n;
nat _k;
subset _actual;
bool _final;
```

Archiu iter_subset.cpp:

```
C/C++
#include "iter_subset.hpp"

/* Pre: Cert
Post: Construeix un iterador sobre els subconjunts de k elements
de {1, ..., n}; si k > n no hi ha res a recórrer. */
iter_subset::iter_subset(nat n, nat k) throw(error) { // cost  $\Theta(k)$ 
    _n = n;
    _k = k;
    _final = false;
    if (k > n) {
        _final = true;
        _actual.clear();
    } else {
        for (nat i = 0; i < k; ++i) {
            _actual.push_back(i + 1);
        }
    }
}

/* Tres grans. Constructor per còpia, operador d'assignació i destructor. */

/* Pre: Cert
Post: Construeix una còpia exacta de l'iterador proporcionat. */
iter_subset::iter_subset(const iter_subset& its) throw(error) { // cost  $\Theta(k)$ 
    _n = its._n;
```



```

    _k = its._k;
    _actual = its._actual;
    _final = its._final;
}

/* Pre: Cert
   Post: Assigna a l'iterador actual una còpia exacta de l'iterador proporcionat. */
iter_subset& iter_subset::operator=(const iter_subset& its) throw(error) { // cost
0(k)
    if (this != &its) {
        _n = its._n;
        _k = its._k;
        _actual = its._actual;
        _final = its._final;
    }
    return *this;
}

/* Pre: Cert
   Post: Destruïx l'iterador alliberant qualsevol recurs associat. */
iter_subset::~iter_subset() throw() {} // cost 0(1)

/* Pre: Cert
   Post: Retorna cert si l'iterador ja ha visitat tots els subconjunts
de k elements presos d'entre n; o dit d'una altra forma, retorna
cert quan l'iterador apunta a un subconjunt sentinella fictici
que queda a continuació de l'últim subconjunt vàlid. */
bool iter_subset::end() const throw() { // cost 0(1)
    return _final;
}

/* Operador de desreferència.
   Pre: Cert
   Post: Retorna el subconjunt apuntat per l'iterador;
   llança un error si l'iterador apunta al sentinella. */
subset iter_subset::operator*() const throw(error) { // cost 0(1)
    if (_final) {
        throw error(IterSubsetIncorr);
    }
    return _actual;
}

/* Operador de preincrement.
   Pre: Cert
   Post: Avança l'iterador al següent subconjunt en la seqüència i el retorna;
   no es produeix l'avançament si l'iterador ja apuntava al sentinella. */
iter_subset& iter_subset::operator++() throw() { // cost 0(k)
    if (_final) {
        return *this;
    }

    subset ultimSubconjunt;
    for (nat i = _n - _k; i < _n; ++i) {
        ultimSubconjunt.push_back(i + 1);
    }
}

```

```

    }

    if (_actual == ultimSubconjunt) {
        _final = true;
    } else {
        for (int i = _k - 1; i >= 0; --i) {
            if (_actual[i] < _n - (_k - 1 - i)) {
                ++_actual[i];
                for (nat j = i + 1; j < _k; ++j) {
                    _actual[j] = _actual[j - 1] + 1;
                }
                return *this;
            }
        }
        _final = true;
    }

    return *this;
}

/*Pre: Cert
Post: Avança l'iterador al següent subconjunt en la seqüència i retorna el seu valor
previ; no es produeix l'avançament si l'iterador ja apuntava al sentinella. */
iter_subset iter_subset::operator++(int) throw() { // cost  $\Theta(k)$ 
    iter_subset temp(*this);
    if (!_final) {
        ++(*this);
    }
    return temp;
}

/* Pre: Cert
Post: Retorna cert si dos iteradors són iguals (apunten al mateix estat). */
bool iter_subset::operator==(const iter_subset& c) const throw() { // cost  $\Theta(k)$ 
    if (_n != c._n and _k == 0 and c._k == 0 and _actual == c._actual and _final ==
        c._final) {
        return true;
    }
    return _n == c._n && _k == c._k && _actual == c._actual && _final == c._final;
}

/* Pre: Cert
Post: Retorna cert si dos iteradors no són iguals. */
bool iter_subset::operator!=(const iter_subset& c) const throw() { // cost  $\Theta(k)$ 
    return !(*this == c);
}

```

4.3 Classe Diccionari Final

4.4 Classe Anagrames Final

En la versió final de la classe Anagrames, vam aconseguir integrar els canvis que havíem provat en les versions anteriors, resolent diversos errors que havíem trobat durant el procés. Un dels principals problemes que vam afrontar va ser la gestió de la secció privada, que causava dificultats en la compilació de la classe. Per solucionar-ho, vam decidir refactorar completament aquesta secció de codi i simplificar la gestió dels elements privats, eliminant els problemes que havíem tingut fins aquell moment.

Després d'alguns intents, vam aconseguir que la classe funcionés de manera més neta, però vam trobar que la taula de dispersió no estava implementada tal com volíem. A la versió 4, vam resoldre molts d'aquests problemes, però la implementació de la taula de dispersió encara no era ideal, ja que no seguia l'estructura que havíem definit a les versions anteriors.

Finalment, en la versió final, vam poder integrar la taula de dispersió de la mateixa manera que a les primeres versions, mantenint la simplicitat aconseguida a la versió 4. Això va permetre resoldre tots els errors anteriors i aconseguir una versió més eficaç i robusta de la classe. La taula de dispersió és ara el cor del diccionari d'anagrames, gestionant les paraules agrupades segons els seus anagrames i permetent una cerca ràpida i eficaç.

En resum, la versió final va ser un punt d'inflexió on vam aconseguir integrar la taula de dispersió de forma correcta, mantenint alhora la simplicitat i la correcció que vam aconseguir a la versió 4. Això ens va permetre obtenir una implementació neta, eficient i funcional de la classe Anagrames.

Archiu anagrames.rep:

```
C/C++
struct node {
    string clau;
    list<string> paraules;
    node* seg;

    node(const string& c) : clau(c), seg(nullptr) {}
};

node** taula;
nat mida;
nat quants;

nat h(const string& clau) const;
void redispersio();
```

Archiu anagrames.cpp:

```

C/C++
#include "anagrames.hpp"
#include "word_toolkit.hpp"

/* Pre: Cert
   Post: Intercanvia els valors de 'a' i 'b'. */
void swap(char& a, char& b) { // cost  $\Theta(1)$ 
    char temporal = a;
    a = b;
    b = temporal;
}

/* Pre: Cert
   Post: Particiona la cadena 's' al voltant del pivot i retorna la seva posició. */
int particio(string& s, int esquerra, int dreta) { // cost  $\Theta(n)$ 
    char pivot = s[dreta];
    int i = esquerra - 1;

    for (int j = esquerra; j < dreta; ++j) {
        if (s[j] <= pivot) {
            ++i;
            swap(s[i], s[j]);
        }
    }
    swap(s[i + 1], s[dreta]); // Col·loca el pivot en la seva posició correcta
    return i + 1;
}

/* Pre: esquerra <= dreta i 's' és una cadena vàlida.
   Post: Ordena la cadena 's' en ordre ascendent entre les posicions 'esquerra' i 'dreta'. */
void quicksort(string& s, int esquerra, int dreta) { // cost  $\Theta(n * \log n)$  promedi,
pitjor cas es cost  $\Theta(n^2)$ 
    if (esquerra < dreta) {
        int pivot = particio(s, esquerra, dreta);
        quicksort(s, esquerra, pivot - 1); // Ordena la part esquerra del pivot
        quicksort(s, pivot + 1, dreta); // Ordena la part dreta del pivot
    }
}

/* Pre: Cert
   Post: Construeix un anagrama buit. */
anagrames::anagrames() throw(error) { // cost  $\Theta(1)$ 
    taula = new node*[101]();
    mida = 0;
    quants = 101;
}

/* Pre: Cert
   Post: Construeix un nou objecte idèntic a 'other'. */
anagrames::anagrames(const anagrames& other) throw(error) { // cost  $\Theta(n)$ 
    taula = new node*[other.quants]();
    mida = other.mida;
    quants = other.quants;
    for (nat i = 0; i < quants; ++i) {

```

```

        if (other.taula[i]) {
            node* actual = other.taula[i];
            node** this_actual = &taula[i];

            while (actual) {
                *this_actual = new node(actual->clau); // Crea un nou node amb la
mateixa clau
                (*this_actual)->paraules = actual->paraules; // Copia la llista de
paraules
                actual = actual->seg;
                this_actual = &((*this_actual)->seg); // Avança al següent node
            }
        }
    }

    /* Pre: Cert
    Post: Assigna a aquest objecte els valors d''other'. */
    anagrames& anagrames::operator=(const anagrames& other) throw(error) { // cost  $\theta(n)$ 
        if (this != &other) {
            for (nat i = 0; i < quants; ++i) {
                node* n = taula[i];
                while (n) {
                    node* temp = n;
                    n = n->seg;
                    delete temp; // Allibera la memòria dels nodes actuals
                }
            }
            delete[] taula;

            taula = new node*[other.quants]();
            mida = other.mida;
            quants = other.quants;

            for (nat i = 0; i < quants; ++i) {
                if (other.taula[i]) {
                    node* actual = other.taula[i];
                    node** this_actual = &taula[i];

                    while (actual) {
                        *this_actual = new node(actual->clau); // Crea un nou node amb
la mateixa clau
                        (*this_actual)->paraules = actual->paraules; // Copia la llista
de paraules
                        actual = actual->seg;
                        this_actual = &((*this_actual)->seg); // Avança al següent node
                    }
                }
            }
            return *this;
        }

        /* Pre: Cert

```

```

    Post: Allibera tota la memòria dinàmica associada a l'objecte. */
anagrames::~~anagrames() { // cost  $\Theta(n)$ 
    for (nat i = 0; i < quants; ++i) {
        node* n = taula[i];
        while (n != nullptr) {
            node* temp = n;
            n = n->seg;
            delete temp; // Allibera la memòria de cada node
        }
        taula[i] = nullptr;
    }
    delete[] taula;
    taula = nullptr; // Allibera la memòria de la taula
}

/* Pre: Cert
    Post: Afegeix la paraula p al diccionari; si la paraula p ja
    formava part del diccionari, l'operació no té cap efecte. */
void anagrames::insereix(const string& p) throw(error) { // cost  $\Theta(p + m)$ 
    diccionari::insereix(p); // Insereix la paraula al diccionari

    string p_ordenada = p;
    quicksort(p_ordenada, 0, p_ordenada.size() - 1); // Ordena la paraula

    nat index = h(p_ordenada);
    node* n = taula[index];

    while (n != nullptr) {
        if (n->clau == p_ordenada) {
            for (list<string>::iterator it = n->paraules.begin(); it !=
n->paraules.end(); ++it) {
                if (*it == p) return; // No afegeix si la paraula ja existeix
            }
            n->paraules.push_back(p); // Afegeix la paraula a la llista
            return;
        }
        n = n->seg;
    }

    node* nou_node = new node(p_ordenada);
    nou_node->paraules.push_back(p); // Afegeix la paraula al nou node
    nou_node->seg = taula[index];
    taula[index] = nou_node;
    ++mida;

    if (mida > quants * 0.75) { // Redistribueix si la taula està plena
        redispersio();
    }
}

/* Pre: Cert
    Post: Retorna la llista de paraules p tals que anagrama_canonic(p)=a ordenades en
    ordre ascendent.
    Llança un error si les lletres de a no estan en ordre ascendent. */

```

```

void anagrames::mateix_anagrama_canonic(const string& a, list<string>& L) const
throw(error) { // cost  $\Theta(L * p + m * L + m * \log L)$ 
    if (!word_toolkit::es_canonic(a)) {
        throw error(NoEsCanonic);
    }

    for (list<string>::iterator it = L.begin(); it != L.end(); it++) {
        string it_ordenada = *it;
        quicksort(it_ordenada, 0, it_ordenada.size() - 1);
        if (it_ordenada != a) {
            it = L.erase(it); // Elimina les paraules que no són anagrames canònics
        } else {
            ++it;
        }
    }

    nat index = h(a);
    node* n = taula[index];

    while (n != nullptr) {
        if (n->clau == a) {
            for (list<string>::iterator it_paraula = n->paraules.begin();
it_paraula != n->paraules.end(); ++it_paraula) {
                bool trobat = false;
                for (list<string>::iterator it_existent = L.begin(); it_existent !=
L.end(); ++it_existent) {
                    if (*it_existent == *it_paraula) {
                        trobat = true;
                        break; // Surt del bucle si es troba la paraula
                    }
                }
                if (!trobat) {
                    L.push_back(*it_paraula); // Afegeix les paraules que no
existeixen
                }
            }
            L.sort(); // Ordena la llista de paraules
            return;
        }
        n = n->seg;
    }
    L.clear(); // Neteja la llista si no es troben anagrames
}

/* Pre: Cert
   Post: Retorna la posició de hash per a la clau donada. */
nat anagrames::h(const string& clau) const { // cost  $\Theta(p)$ 
    nat hash = 0;
    for (nat i = 0; i < clau.size(); ++i) {
        hash = (hash * 31 + clau[i]) % quants; // Calcula l'índex de hash
    }
    return hash;
}

```

```

/* Pre: Cert
   Post: Redistribueix els elements de la taula hash en una nova taula de mida doble.
*/
void anagrames::redispersio() { // cost  $\Theta(n + m * p)$ 
    nat nou_quants = quants * 2;
    node** nova_taula = new node*[nou_quants]();

    for (nat i = 0; i < quants; ++i) {
        node* n = taula[i];
        while (n != nullptr) {
            node* temp = n;
            n = n->seg;

            nat nou_index = 0;
            for (size_t j = 0; j < temp->clau.size(); ++j) {
                nou_index = (nou_index * 31 + temp->clau[j]) % nou_quants; // Calcula
                el nou index de hash
            }

            temp->seg = nova_taula[nou_index];
            nova_taula[nou_index] = temp; // Reubica el node en la nova taula
        }
    }

    delete[] taula;
    taula = nova_taula;
    quants = nou_quants; // Actualitza la mida de la taula
}

```

4.5 Classe Obte_Paraules Final

5. Conclusions i Lliçons Apreses

Durant aquest projecte, vam adonar-nos de la importància d'una bona planificació des de l'inici per evitar complicacions a mesura que avançàvem. Una de les primeres lliçons va ser la necessitat de tenir un disseny clar i ben estructurat, especialment quan es treballa amb estructures de dades complexes com taules de dispersió. Això va ajudar a evitar errors i va facilitar la integració de la taula de dispersió de manera eficient en les etapes posteriors del projecte.

A mesura que avançàvem, vam anar simplificant la classe, optimitzant operacions com la cerca, inserció i manipulació dels elements de la taula de dispersió. Ens vam adonar que, a vegades, fer una refactorització a mig projecte pot ser clau per millorar el rendiment del codi i mantenir-lo net i entenedor. A més, les operacions de dispersió es van convertir en un element central, on vam aconseguir un bon equilibri entre complexitat i rendiment.

La gestió de la memòria també va ser un aspecte clau que vam haver de gestionar amb cura. Ens vam trobar amb alguns desafiaments relacionats amb l'assignació i alliberament de memòria, però, finalment, vam aconseguir implementar una gestió eficient que va millorar la robustesa del codi.

Una altra lliçó important va ser la importància de realitzar proves constants i de fer-les en diferents etapes del projecte. Les proves ens van permetre identificar problemes de manera ràpida i van afavorir la comprensió global del codi. En aquest sentit, la importància de tenir un conjunt de proves robustes que cobreixin tots els possibles casos va ser clau per garantir que el sistema fos fiable i establert en tot moment.

En resum, aquest projecte ens ha ensenyat diverses lliçons sobre la importància de la planificació, l'optimització, la gestió eficient de la memòria i la realització de proves



continuades. Aquestes experiències ens seran molt útils en futurs projectes, on esperem aplicar aquests coneixements per aconseguir resultats més ràpids i de més qualitat.